

Table of contents

Table of contents.....	1
Shneiderman's Eight Golden Rules of Interface Design.....	2
1. Strive for Consistency.....	29
2. Enable Frequent Users to Use Shortcuts.....	2
3. Offer Informative Feedback.....	3
4. Design Dialogs to Yield Closure.....	3
5. Offer Error Prevention and Simple Error Handling.....	3
6. Permit Easy Reversal of Actions.....	3
7. Support Internal Locus of Control.....	3
8. Reduce Short-term Memory Load.....	4
Nielsen's ten heuristics.....	4
1. Visibility of system status.....	4
2. Match between system and the real world.....	4
3. User control and freedom.....	4
4. Consistency and standards.....	4
5. Error prevention.....	4
6. Recognition rather than recall.....	5
7. Flexibility and efficiency of use.....	5
8. Aesthetic and minimalist design.....	5
9. Help users recognize, diagnose and recover from errors.....	5
10. Help and documentation.....	5
Norman's 7 model of interaction.....	6
1. Establishing the goal.....	6
2. Forming the intention.....	6
3. Specifying the action sequence.....	6
4. Executing the action.....	7
5. Perceiving the system state.....	7
6. Interpreting the system state.....	7
7. Evaluating the system state with respect to the goals and intentions.....	7
How the Execution-Evaluation Cycle works in interaction:.....	7
Cognitive Walkthrough.....	8
abowd and beale framework.....	10
Example Exam Question.....	10
Errors.....	11
understanding user(persona, scenarios, cultural probes).....	12
interaction design and interdisciplinary fields.....	13
Human Memory (short term, long term, sensory).....	13
Reasoning.....	14
Manipulation.....	15

Human I/O channels.....	16
Design Laws.....	16
Fitts' Law.....	16
Hick's Law.....	16
Interaction modes.....	17
usability principles.....	18
Predictability.....	18
2. Synthesizability.....	18
3. Familiarity.....	18
4. Generalizability.....	19
5. Consistency.....	19
. Observability.....	20
Definition: The ability of the user to evaluate the internal state of the system from its perceivable representation.....	20
• HCI Example: In a file downloading interface, the user can see the progress through a visual progress bar. This allows the user to observe how much of the file has been downloaded and estimate the remaining time. This transparency enables users to understand the system's current state.....	20
Task Conformance.....	21
usability standards.....	22
top 10 mistakes in design.....	23
Prototyping.....	24
Types of Prototyping:.....	24
Advantages of Prototyping:.....	24
Related Concepts:.....	25
Golden Ratio.....	25

Shneiderman's Eight Golden Rules of Interface Design

Mnemonic: **CFSDERL**

1. Strive for Consistency

Explanation: Consistency in actions, layout, terminology, and command use helps users develop a clear understanding of how to interact with the system.

Example: A word processor consistently uses the same keyboard shortcuts (like Ctrl+C for copy and Ctrl+V for paste) throughout the application. This familiarity reduces the learning curve for users.

2. Enable Frequent Users to Use Shortcuts

Explanation: Providing shortcuts allows experienced users to perform familiar actions quickly, enhancing efficiency.

Example: In graphic design software, users can use keyboard shortcuts (like B for the brush tool or E for the eraser) to switch tools instantly instead of navigating through menus.

3. Offer Informative Feedback

Explanation: Providing feedback for every user action reassures users that their inputs have been received and understood, making the system more intuitive.

Example: When a user submits a form online, a confirmation message appears, such as “Your submission has been received,” indicating successful completion.

4. Design Dialogs to Yield Closure

Explanation: Users should know when they have completed a task. Clear indications of task completion provide a sense of accomplishment and reduce confusion.

Example: After completing an online purchase, the user is presented with a “Thank you for your order” message, along with order details, signaling that the transaction is finished.

5. Offer Error Prevention and Simple Error Handling

Explanation: Systems should be designed to prevent errors and provide clear instructions for recovery when they occur.

Example: A file deletion dialog asks the user to confirm their action (e.g., “Are you sure you want to delete this file?”), preventing accidental deletions and providing a clear way to cancel.

6. Permit Easy Reversal of Actions

Explanation: Allowing users to easily undo actions helps reduce anxiety and encourages exploration, knowing they can return to the previous state.

Example: A text editor allows users to use Ctrl+Z to undo recent changes. This feature encourages users to experiment with formatting and editing without fear of making irreversible mistakes.

7. Support Internal Locus of Control

Explanation: Users should feel in control of the system, which should respond to their actions in a predictable manner.

Example: A music player app allows users to create playlists and control playback, giving them the ability to choose what music to listen to and when.

8. Reduce Short-term Memory Load

Explanation: Keep displays simple and consolidate information to reduce the cognitive burden on users, making it easier to process information.

Example: A mobile app provides a dashboard that consolidates relevant information (e.g., recent notifications, messages, and calendar events) into one screen, minimizing the need to navigate through multiple pages.

Nielsen's ten heuristics

Mnemonic: CURVE-MFAHH

1. Visibility of system status

Always keep users informed about what is going on, through appropriate feedback within reasonable time. For example, if a system operation will take some time, give an indication of how long and how much is complete.

2. Match between system and the real world

The system should speak the user's language, with words, phrases and concepts familiar to the user, rather than system-oriented terms. Follow real-world conventions, making information appear in natural and logical order.

3. User control and freedom

Users often choose system functions by mistake and need a clearly marked 'emergency exit' to leave the unwanted state without having to go through an extended dialog. Support undo and redo.

4. Consistency and standards

Users should not have to wonder whether words, situations or actions mean the same thing in different contexts. Follow platform conventions and accepted standards.

5. Error prevention

Make it difficult to make errors. Even better than good error messages is a careful design that prevents a problem from occurring in the first place.

6. Recognition rather than recall

Make objects, actions and options visible. The user should not have to remember information from one part of the dialog to another. Instructions for use of the system should be visible or easily retrievable whenever appropriate.

7. Flexibility and efficiency of use

Allow users to tailor frequent actions. Accelerators – unseen by the novice user – may often speed up the interaction for the expert user to such an extent that the system can cater to both inexperienced and experienced users.

8. Aesthetic and minimalist design

Dialogs should not contain information that is irrelevant or rarely needed. Every extra unit of information in a dialog competes with the relevant units of information and diminishes their relative visibility.

9. Help users recognize, diagnose and recover from errors

Error messages should be expressed in plain language (no codes), precisely indicate the problem, and constructively suggest a solution.

10. Help and documentation

Few systems can be used with no instructions so it may be necessary to provide help and documentation. Any such information should be easy to search, focussed on the user's task, list concrete steps to be carried out, and not be too large.

Norman's 7 model of interaction

Mnemonic: EFSE-PIE

Donald Norman's Interaction framework

- user establishes the goal
- formulates intention
- specifies actions at interface
- executes action
- perceives system state
- interprets system state
- evaluates system state with respect to goal

Some systems are harder to use than others

- Gulf of Execution - user's formulation of actions may be different to those actions allowed by the system
- Gulf of Evaluation - user's expectation of the changed system state may be different to the actual presentation of this state

Norman's model concentrates on user's view of the interface only

outlines how humans interact with computers in a seven-stage cycle, divided into two main phases: **execution** (planning and performing actions) and **evaluation** (observing and interpreting results). Here's a breakdown of each stage with examples:

1. Establishing the goal

- **Explanation:** The user starts by deciding on a high-level goal, which is what they ultimately want to achieve.
- **Example:** A user wants to send an email to confirm a meeting. The goal is "send an email."

2. Forming the intention

- **Explanation:** The user forms an intention, which is a more specific plan on how to achieve the goal.
- **Example:** The user decides to open their Gmail account, compose a new email, and type the meeting confirmation message.

3. Specifying the action sequence

- **Explanation:** The user identifies the specific steps they need to take to execute their intention.
- **Example:** The user plans to:
 1. Open a web browser.
 2. Navigate to the Gmail website.
 3. Log in to their account.

4. Click "Compose."
5. Enter the recipient's email, type the message, and click "Send."

4. Executing the action

- **Explanation:** The user physically performs the specified actions by interacting with the computer interface.
- **Example:** The user opens the browser, goes to Gmail, logs in, writes the email, and presses "Send."

5. Perceiving the system state

- **Explanation:** After the action is executed, the user observes the computer's response to determine what has happened.
- **Example:** The user sees the confirmation message that the email was sent successfully.

6. Interpreting the system state

- **Explanation:** The user interprets the system's feedback to understand whether the action has succeeded or not.
- **Example:** The user reads the "Your message has been sent" notification and confirms that the email was indeed sent.

7. Evaluating the system state with respect to the goals and intentions

- **Explanation:** The user compares the system's response to their original goal and intention to see if the outcome is as expected.
- **Example:** The user evaluates that the email was sent successfully, which means the goal of confirming the meeting was achieved.

How the Execution-Evaluation Cycle works in interaction:

1. **Execution phase** (steps 1-4): The user decides what they want to do (set a goal), how to do it (form an intention and action sequence), and then performs the action.
2. **Evaluation phase** (steps 5-7): The user observes the result (perceives the system state), interprets what happened, and checks if the outcome meets their goal.

Another Example: Uploading a Photo on Instagram

Cognitive Walkthrough

ex 9.3.1

Cognitive walkthrough.

day / date:

* Usability evaluation method designed to assess how easily a new user can learn to use a system through exploration.
(like software code walkthrough).

Process.

* INPUTS.

- ① Prototype or specification of the system.
- ② Task desc → to show something typical that users would do.
- ③ Complete action list → steps with details to complete the task using the system.
- ④ User profile → user exp + knowledge levels.

* EVALUATION PROCESS.

- ① Does the action lead to the user's goal at that pt?
- ② Can the user see the action is available? (e.g. button visible?)
- ③ Will user recognize the action as needed?

→ did it do what the user wanted?

- ④ Will user understand the feedback after completing the task?

* DOCUMENTATION

↳ Each action step is documented w/ its corresponding answers to 4qs.

↳ If answers indicate usability problems, they are recorded in usability problem report.

Eg Designing a remote control for a VCR to do timed recordings.

Action List
① UA 1 : Press the timed record button
SD 1 : (response) Display moves to timer mode. Flashing cursor appears after start.

② UA 1 → Q1, 2, 3, 4.

③ Find the usability problem.

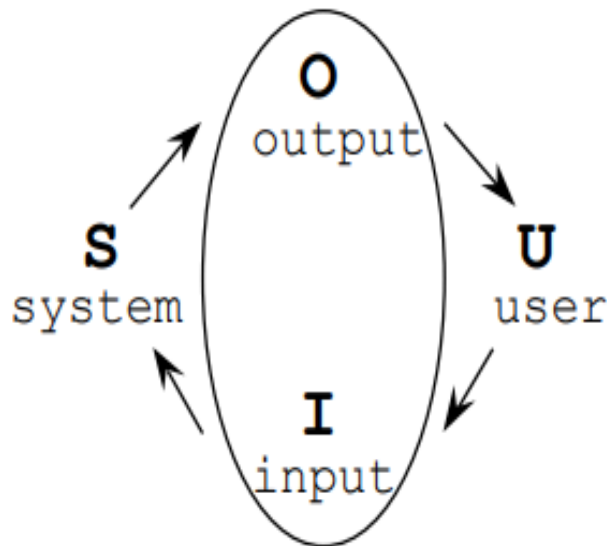
④ Can your target audience resolve this usability problem/get over it?

abowd and beale framework

Extended by Abowd and Beale:

their interaction framework has 4 parts

- user
- input
- system
- output

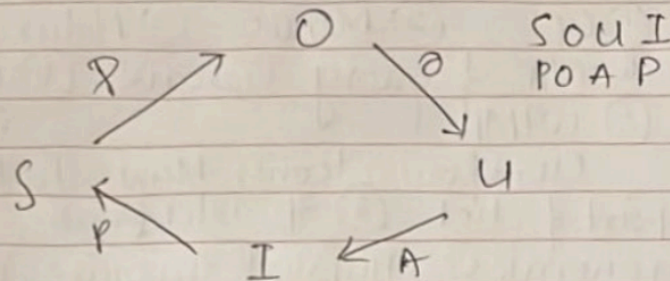


Each has its own unique language. Interaction necessitates translation between languages; problems in interaction occur when translation between one language and the next is difficult, or impossible.

User intentions translated into actions at the interface, translated into alterations of system state, which in turn are reflected in the output display, which is interpreted by the user.

day / date:

Abowd and Beale Framework



Φ : presentation

- * Feedback to user
- * Showing system state after input.

O: observation

- * Users interp of the system state output to determine if the goal has been achieved or not

A: articulation

- * Translation of user's goals into actions (pressing a button)

p: performance

- * Systems interp of the user's input into actions that change the system state

Scenario: A user interacts with a mobile banking app.

- **User:** The user is a bank customer who wants to check their account balance and transfer funds.

- **Input:** The user taps on the app icon to open it, enters their credentials using the touchscreen keyboard, and selects the options for viewing the balance and making a transfer.
- **System:** The app processes the input by authenticating the user, fetching account information from the bank's server, and preparing the transfer function.
- **Output:** The app displays the user's account balance and provides a confirmation message for the fund transfer. It might also send a push notification confirming the transaction.

Example Exam Question

Question: Describe the components of the Abowd and Beale Interaction Framework (User, Input, System, Output) and provide an example of how they apply to a specific application in HCI.

Answer Outline:

- **User:** Define the target user and their needs.
- **Input:** Explain the input methods available in the application.
- **System:** Describe the main functionalities of the application.
- **Output:** Discuss how the system provides feedback to the user and supports their goals.

Interaction Example:

1. User's Goal

The user wants to play a song from their music library.

2. Input

The user taps on the song title (input event) or says "play [song name]" using voice control.

3. System Process

The music app receives the tap or voice input, searches its database for the song, and sends a command to the media player to start playing the song.

4. Output

The system displays the song's title and artist on the screen, begins playing the song through the phone's speakers, and updates the progress bar to show how much of the song has played.

Breakdown According to the Framework:

Articulation: The user's intent to play a song is articulated through a physical action (tap) or a vocal command.

Performance: The system interprets the input and processes it.

Presentation: The system presents feedback by showing the song playing and providing audio output.

Observation: The user observes the song title and listens to the audio, confirming their action was successful.

Errors

Human error – slips and mistakes

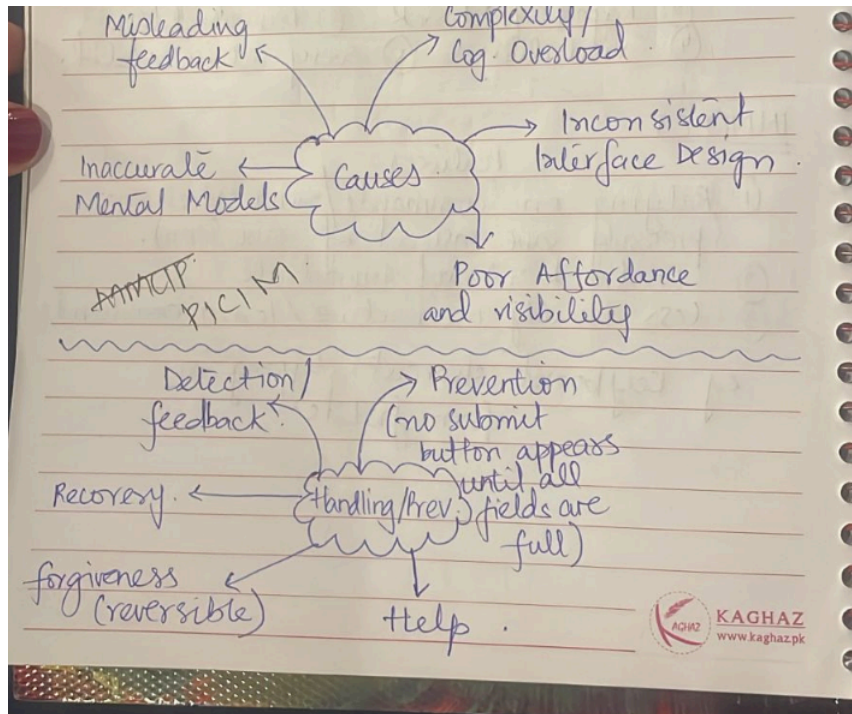


Human errors are often classified into *slips* and *mistakes*. We can distinguish these using Norman's gulf of execution.

If you understand a system well you may know exactly what to do to satisfy your goals – you have formulated the correct action. However, perhaps you mistype or you accidentally press the mouse button at the wrong time. These are called *slips*; you have formulated the right action, but fail to execute that action correctly.

However, if you don't know the system well you may not even formulate the right goal. For example, you may think that the magnifying glass icon is the 'find' function, but in fact it is to magnify the text. This is called a *mistake*.

If we discover that an interface is leading to errors it is important to understand whether they are slips or mistakes. Slips may be corrected by, for instance, better screen design, perhaps putting more space between buttons. However, mistakes need users to have a better understanding of the systems, so will require far more radical redesign or improved training, perhaps a totally different metaphor for use.



understanding user(persona, scenarios, cultural probes)

Understanding Users (persona, scenarios, cultural probes)
 show your users - base design focuses on users
 stake holders - directly or indirectly affected by the system
 find out users
 1. who are they?
 a. Probably not like you.
 2. Talk to them.
 participatory design - involve users throughout the design process
 people can tell how things really happen rather than how it SHOULD!
 4. Watch them.
 Cultural Probes: used as one way to gather rich views of an area without
 intrusion. These were originally developed as prompts for
 design but also have been adapted as an added method
 for ethnography. (pg 170)
 Small packet of items designed to provoke & record comments in
 various ways - give people to take away & open in own environment.
 A qualitative way to gather insights into user's lives through diaries,
 photos etc to inform design decisions.
 Persona: fictional characters representing a group of users with similar
 goals & behaviours (rich description) - represents real users
 don't stop at observation go back and discuss with users.
 ↳ what they do? ↳ why they do?
 5. Use your imagination (easier & hard to get time - users)
 Scenarios: stories for design - rich stories for interaction
 augmented by sketches, simulated screen shots etc
 script to act out potential patterns of use
 force you to think about the design in detail & notice potential
 problems before they happen.

User (persona, scenarios, culture probes) day/ date:

① Persona

- Demographic (Age, gender, profession, edu, etc.)
- Behaviors
- Goals
- Frustrations
- Motivations

② Scenario: how persona interacts with the system.

Types → Interaction

Goal-oriented Task-oriented

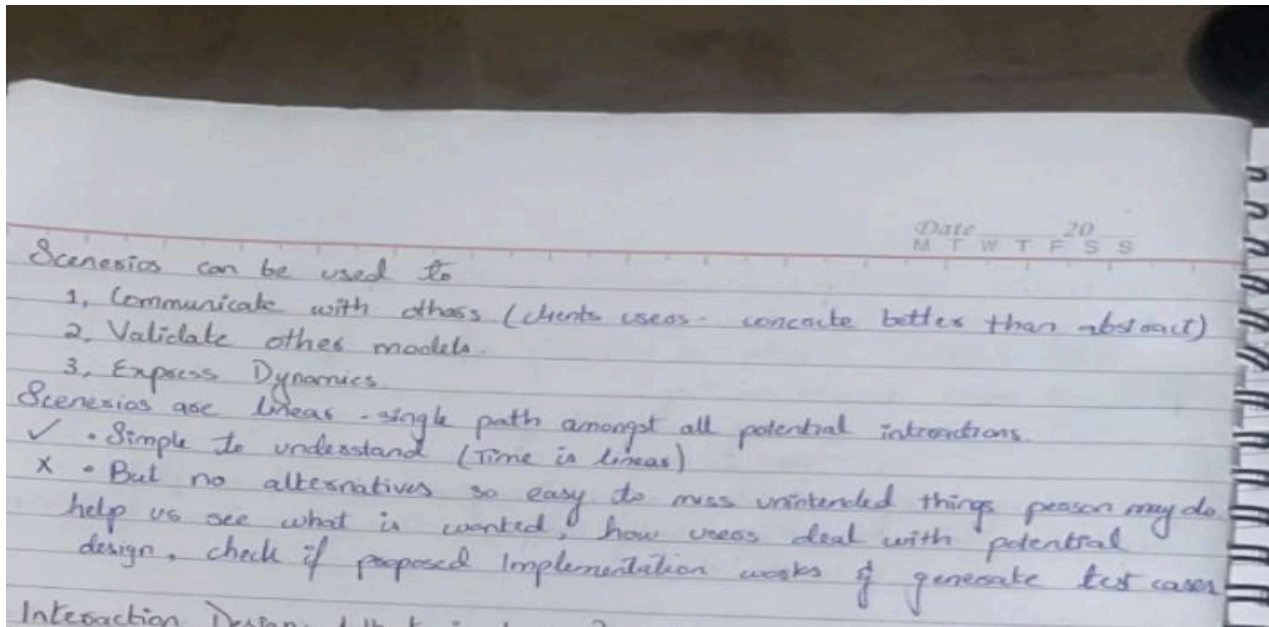
Components

- Persona
- Context (where, when)
- Tasks
- Steps
- Expected outcomes

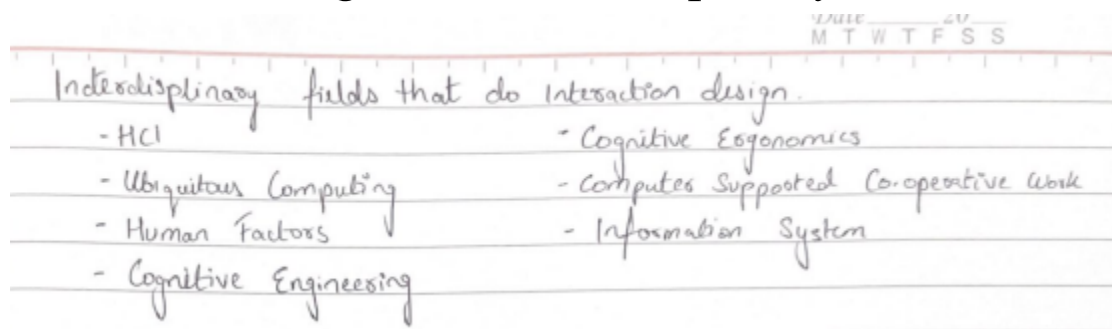
③ Cultural Probes

- * Participatory
- * Users given tools to record aspects of their lives.

KAGHAZ www.kaghaz.pk



interaction design and interdisciplinary fields



Human Memory (short term, long term, sensory)

Sensory Memory:

- Sensory memory acts like a short-term storage for information from our senses (sight, sound, touch).
- For example, when you move your mouse across the screen, you still "see" its trail for a moment because of iconic memory (visual memory). Similarly, echoic memory (sound memory) allows you to briefly recall a notification sound even after it's gone.
- **iconic memory:** Interfaces benefit from visuals that are easy to scan and recognize in milliseconds (e.g., app icons or navigation cues).

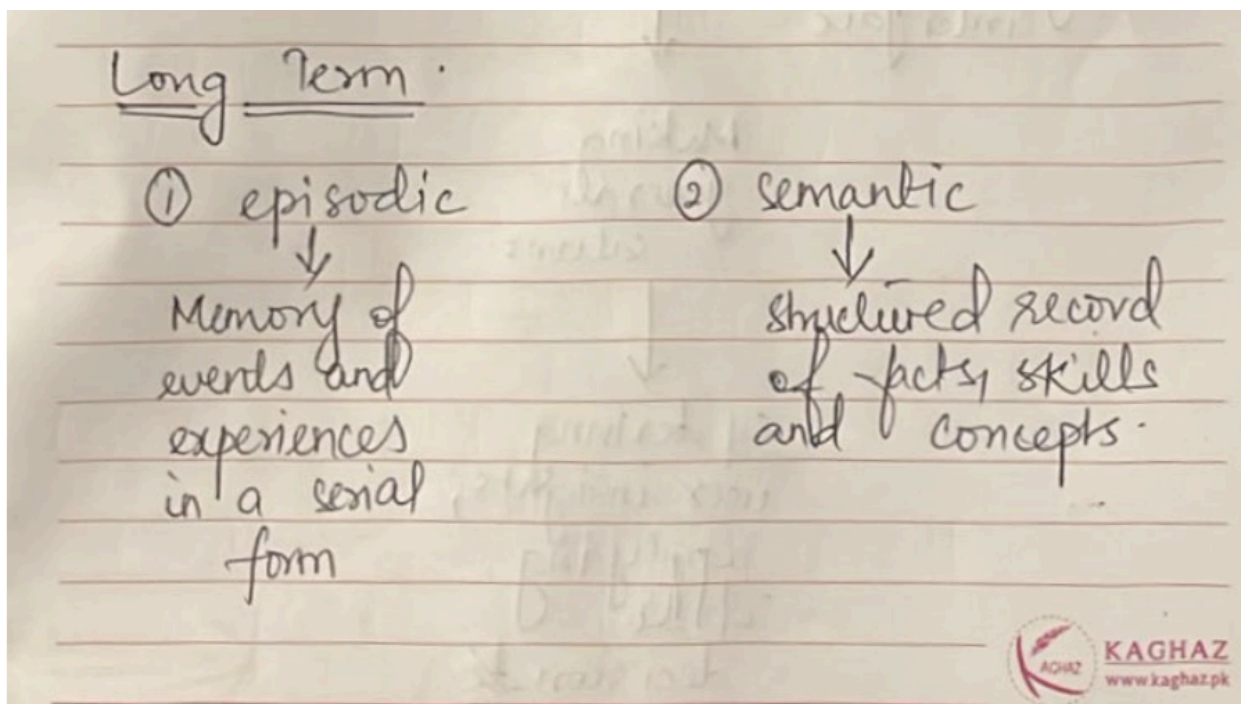
- **Echoic memory:** Audible feedback, like notification sounds, needs to be concise but memorable within the brief time span that auditory information is retained.
- **Haptic memory:** Tactile feedback in devices, such as vibrations in smartphones, gives users immediate confirmation of their actions.

Short-Term Memory:

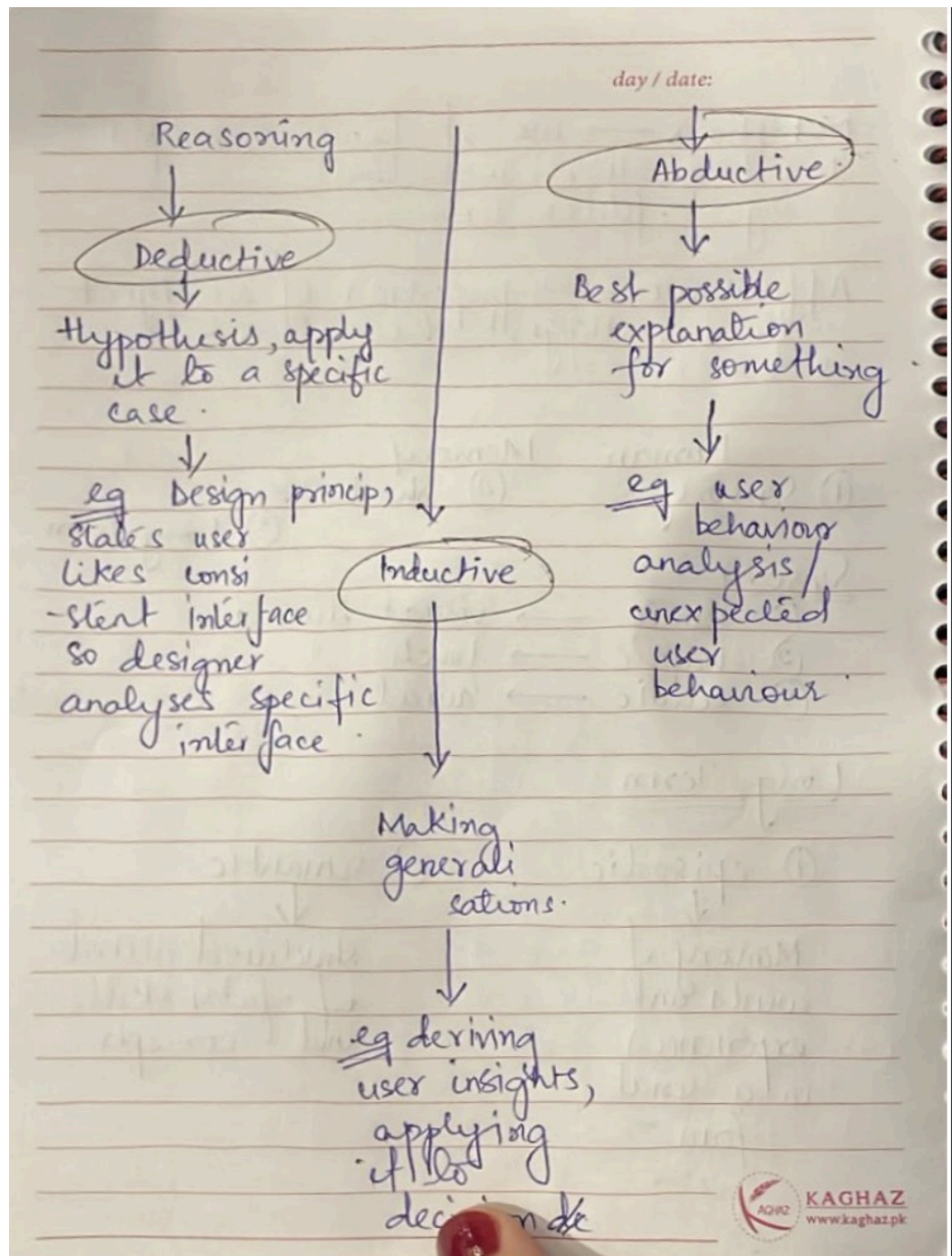
- Short-term memory holds information temporarily. For instance, when typing a password or remembering a sequence of steps in a task, this memory helps you complete those actions.
- It's limited in capacity—typically only about 7 items can be remembered. To improve this, chunking (grouping items) is useful. For example, in user interfaces, breaking complex instructions into smaller, digestible parts helps users process and remember tasks.

Long-Term Memory:

- Long-term memory stores information for a long time. For example, you remember how to navigate a frequently used app or how certain functions work, like dragging files to a recycle bin.
- Users learn through repeated exposure (rehearsal). For instance, if a user repeatedly uses certain shortcuts or patterns in an app, they eventually store them in long-term memory for future use.



Reasoning



Manipulation

Manipulation.

day / date:

DIRECT.

Features of DM Interface

- ① Visibility of objects of interest.
- ② Incremental action with rapid feedback.
- ③ Reversibility
- ④ Syntactic correctness
- ⑤ Replacing complex command languages with direct interaction.



Simplified: ① Drag & drop (instead of commands)
② Rapid feedback ③ Reversibility
④ visibility ⑤ every action is valid.

INDIRECT.

Features:

- ① Relying on commands / menu selection.
(pressing save instead of save icon).
- ② Feedback is not immediate.
- ③ Less visually interactive / learn commands.
eg keyboard shortcuts, typing in form fields.

Human I/O channels

In human–computer interaction, sight, hearing, and touch are vital for effective communication and feedback.

- **Sight** is the primary sense for interacting with systems, allowing users to see visual elements like screens and interfaces. However, it's important to design for users with visual impairments, offering alternative feedback methods like audio or tactile cues.
- **Hearing** complements sight by providing audio feedback, such as alerts or voice commands, and is especially important for users with limited vision.
- **Touch** (haptic perception) provides physical feedback, like feeling buttons depress or sensing object manipulation in virtual environments. It's crucial for users with impairments in other senses and enhances precision in tasks like typing.

Design Laws

Fitts' Law

The formula for Fitts' Law is:

This is formalized in Fitts' law [135]. There are many variations of this formula, which have varying constants, but they are all very similar.

One common form is Movement time = $a + b \log_2(\text{distance}/\text{size} + 1)$

Hick's Law

The formula for Hick's Law is:

$$RT = a + b \cdot \log_2(n + 1)$$

~ Hicks law in slides is: $RT = a + b \log_2(n)$

Design rules

Nielsen's 10 Heuristics (see Chapter 9)

- –Shneiderman's 8 Golden Rules

- -Norman's 7 Principles

Four golden rules

four golden rules of design
 where are you
 what you can do
 where you have been
 where you can go

Interaction modes

mnemonic : **RICEM**

Monday
 day / date: 23rd Sept

HCI

Interaction Modes: Eg Interfaces:

- * Instructing → click
- * Conversing → textbox, searching, chatbot, etc.
- * Responding
- * Explaining
- * Manipulating

Achieving goal within constraints

Different types of constraints → hardware, user, etc.

- * Personas, cultural, discovers (ethnography)
 - ↳ features of user
 - ↳ collecting info using tools.

usability principles

Learnability — the ease with which new users can begin effective interaction and achieve maximal performance.

Mnemonic: FPGMSC

Predictability

- **Definition:** This principle ensures that users can anticipate the effects of their actions based on past experiences with the system.
 - **Example:** In a word processor, when a user clicks the “Save” button, they expect the document to be saved based on previous experience. The system’s consistent response to this action helps users predict what will happen each time they click the “Save” button.
 - **Related Principle:** *Operation visibility* — the system makes it clear what each action will do before the user performs it.
-

2. Synthesizability

- **Definition:** This principle helps users assess the effects of past operations on the current state of the system, either immediately or after some time.
 - **Example:** In design software, if a user deletes an element from a drawing, a dialog box showing the list of recent changes helps the user see the impact of their action immediately. This provides clarity on how their last action affected the current state of the design.
 - **Related Principle:** *Immediate/eventual honesty* — systems should either immediately or eventually reflect the outcome of user actions.
-

3. Familiarity

- **Definition:** Familiarity is the extent to which users can transfer knowledge from real-world experiences or other software when learning to use a new system.

- **Example:** In a digital calculator app, users can quickly understand how to use the buttons because they mimic a physical calculator they've used before. This familiarity reduces the learning curve.
 - **Related Principle:** *Guessability, affordance* — interfaces should suggest their use based on past experiences.
-

4. Generalizability

- **Definition:** This principle supports users in applying knowledge gained from specific interactions to similar situations within the same system or across different systems.
 - **Example:** In a photo editing software, once a user learns how to apply filters to one image, they can generalize that knowledge to apply filters to other images in different parts of the software or even in other similar apps.
 - **Related Principle:** None explicitly, but this principle is closely tied to *consistency*.
-

5. Consistency

- **Definition:** This principle ensures that the input–output behavior is consistent across similar tasks or situations, helping users form correct expectations.
- **Example:** In an operating system, the "Ctrl+C" shortcut consistently works across all applications (Word, Excel, Browser, etc.) to copy text. This uniformity ensures users don't have to relearn the action in different contexts.
- **Related Principle:** None explicitly, but it works alongside *generalizability* to support learnability.

6. Metaphors

A metaphor in design is a figure of speech that conveys an idea or concept by associating it with another, often more familiar one. In HCI, this means using terms, symbols, or visual representations that evoke mental images and associations, making the interface more relatable and easier to navigate

Desktop Metaphor:

- **Example:** The desktop interface of a computer resembles a physical office desk, with files represented as folders and documents as papers. This familiar concept helps users understand where to find and organize their digital files.

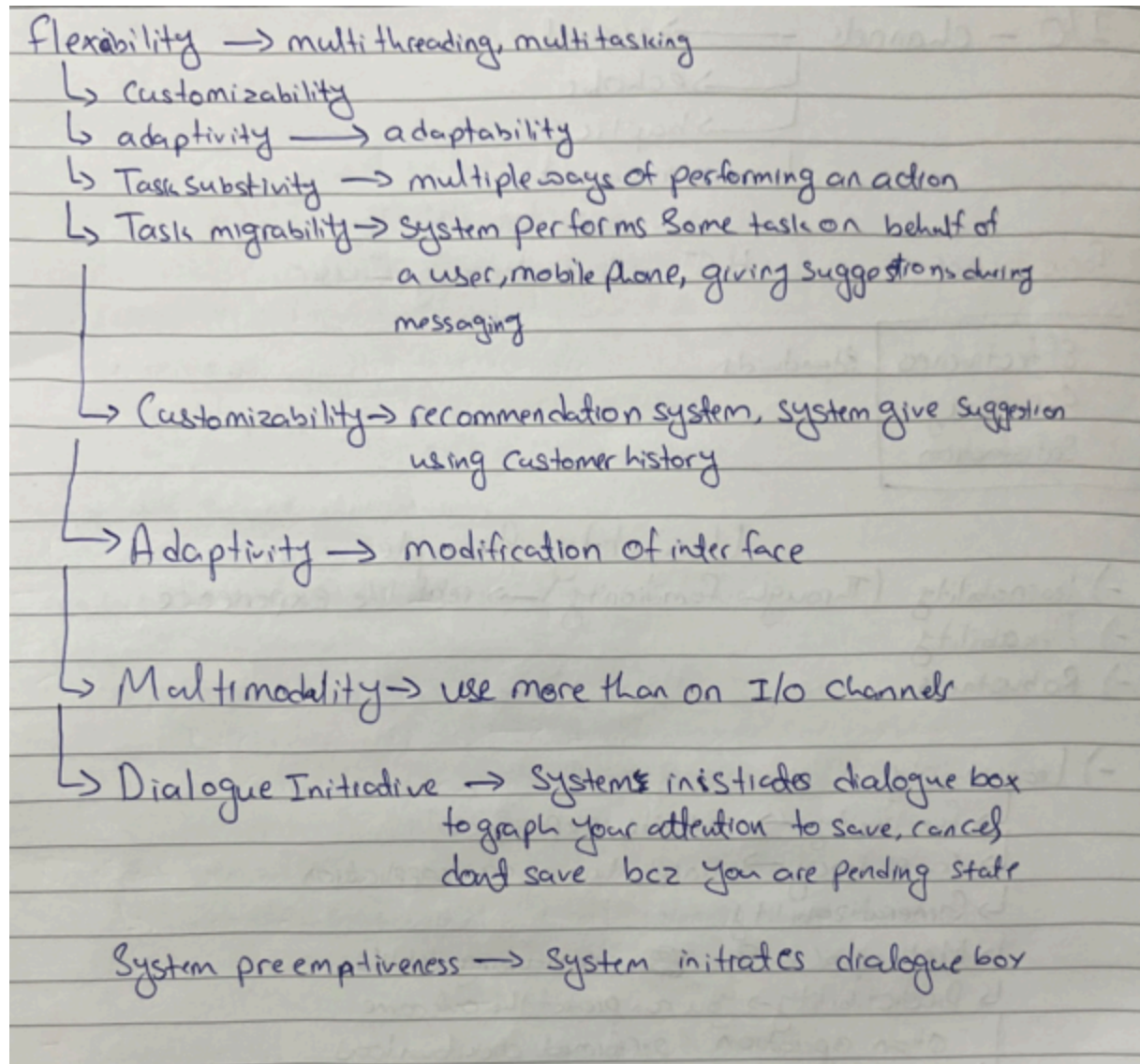
Trash Can Metaphor:

- **Example:** The trash can icon represents the action of deleting files. Users understand that dragging files to the trash can icon will remove them, similar to throwing away physical documents.

Shopping Cart Metaphor:

- **Example:** E-commerce websites use a shopping cart icon to represent the user's selected products. This metaphor draws on the experience of physical shopping, making it easier for users to understand the process of adding items to their purchase.

Flexibility



Robustness

Observability

Definition: The ability of the user to evaluate the internal state of the system from its perceivable representation.

- **HCI Example:** In a file downloading interface, the user can see the progress through a visual progress bar. This allows the user to observe how much of the file has been downloaded and estimate the remaining time. This transparency enables users to understand the system's current state.

> How easily can you achieve the goal? operation visibility

Task Conformance

Definition: The degree to which the system services support all of the tasks the user wishes to perform and in the way that the user understands them.

- **HCI Example:** In a photo editing software like Adobe Photoshop, the tools provided (e.g., crop, resize, color adjustment) conform to what users expect when editing images. The system supports a wide variety of image editing tasks and presents them in ways familiar to users, making them easy to find and use.

Task Conformance
↳ Task adequacy → change the colour of the button in remote power btn
↳ understanding task
↳ Task completeness → goal oriented

Browsability: The user should be able to explore the system easily, like browsing menu options in a software toolbar.

> Browsability → use index in a book / table of content

Persistence: User preferences should be maintained across sessions (e.g., saved login states or remembered preferences in apps).

Persistence → until user does not interact with the system, info is not conveyed to the user keep the info highlighted
keep in notification, keep it bold, unread mail in Gmail

Reachability: Important actions or recovery options should be easy to find, like the undo/redo buttons.

Reachability → when we click on the topic in table of content
browsability makes reachability easy, ease of doing
Something or Search page number of top search

Recoverability

Definition: The ability of the user to take corrective action once an error has been recognized.

- **HCI Example:** In a document editor like Google Docs, users can undo or redo actions using keyboard shortcuts (Ctrl+Z, Ctrl+Y), allowing them to quickly recover from mistakes. Additionally, autosave functionality lets users recover unsaved work if the system crashes.

Recoverability → take on action towards the goal, if you
take a wrong step you can undo easily
Ctrl+Z

usability standards

* Efficiency → time constraints
* Effective → jis problem kai liye banaya woh hai
* Satisfaction

- the process of interaction design

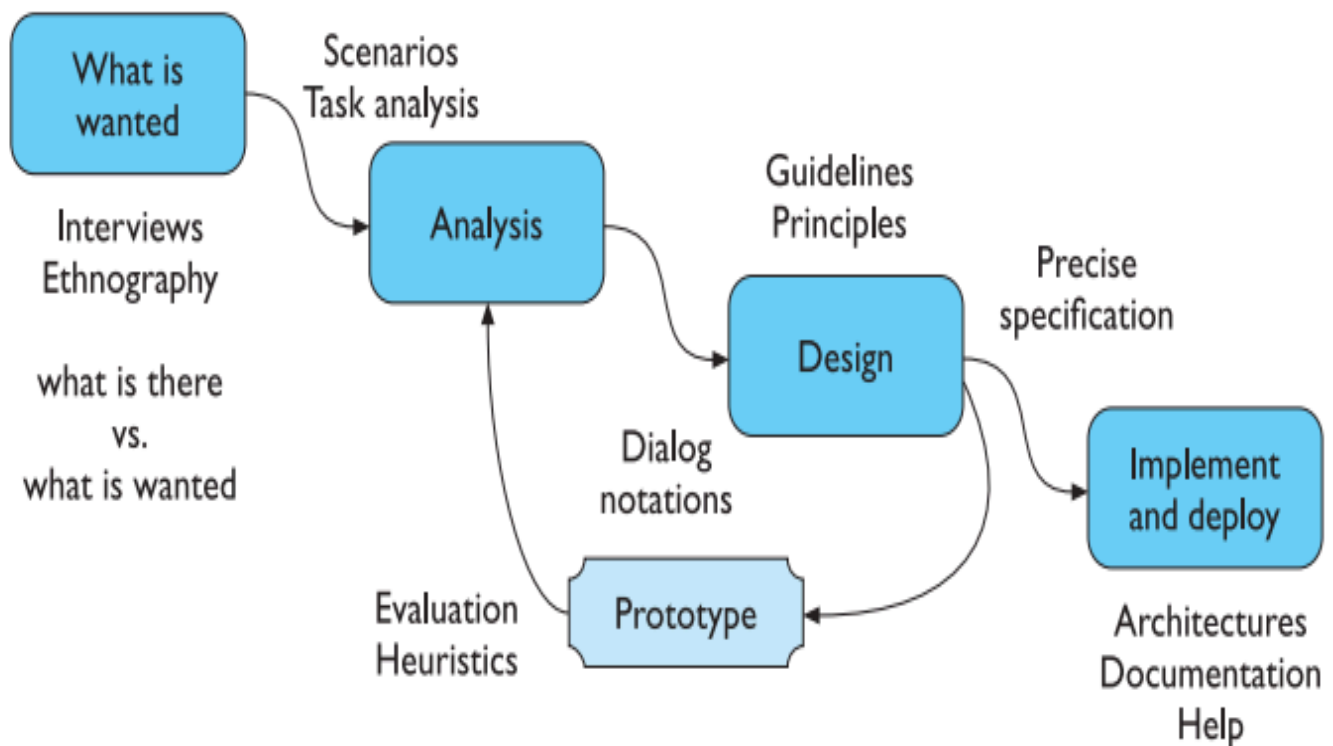


Figure 5.1 Interaction design process

Conceptualizing Interaction Design? No, this is Summary. →

- 1, User centered design user research, Personas, user scenarios, probes ^{cultural}
- 2, Understanding Context of use, physical env, task env, user characteristics (skills) ^{limits}
- 3, Design Principles Affordance, Feedback, Consistency, Simplicity
- 4, Interaction Models Direct Manipulation, Command Line, Natural Uses
- 5, Prototyping & Iteration Iterative, hi-fi & lo-fi (voice, touch etc)
- 6, Evaluating & Testing Usability Testing
- 7, Interdisciplinary Approach Psychology, cognitive science, Graphics
- 8, Emerging Technologies

Page #

human factors

Benefits of Conceptualizing:

- Orientation ^{conceptual questions}
- Open-minded ^{No narrow focus early}
- Common ground ^{establish set of common agreed terms}

A conceptual model provides a working strategy & framework of general concepts and their intentions

Metaphors & analogies
Relationship & Mapping btw concepts } Components

1. Formulate a Conceptual Model

INTERACTION TYPES

- Instructing ^{quick} Issuing commands & selecting option point, save, undo
- Conversing Conversation with system timetable, search engine, chatbot
- Manipulating Interacting in a virtual or physical space dragging & selecting
- Exploring Moving through a virtual environment as a physical space
- Responding System initiates & user chooses whether to respond
Notification

Interaction Styles

- CLI
- menus
- Natural language
- Q/A query dialogue
- form-fill & spreadsheet
- WIMP
- point & click
- 3D interface

Direct Manipulation: • Continuous representation of objects & actions.

- No need to issue ^{complex} commands just press
- Rapid reversible action w immediate feedback

Good for tasks like example: designing, drawing, flying etc.

Interaction type is a description of what the user is doing when interacting with a system for example instructing telling browsing or responding. Interface style is kind of interface used to support the interaction for instance, command, menu-based, gesture as

Interface styles ~

Command	form-fill in	Web	Gestures
Speech	Query	Pen	
Data-entry	Graphical	Augmented reality	

Conceptual ~~that~~ Knowledge that is used to inform design & guide research include.

1. Paradigms Inspiration for a conceptual model

Shared assumptions, concepts values, computing in wild

- Ubiquitous Computing
- Pervasive computing
- Wearable Computing
- Internet of things

2. Visions driving force that frames research & development provide concrete scenarios of how society can use the next generation of imagined technologies.

3. Theory Explanation of a phenomenon.

4. Model simplification of an HCI phenomenon.

enables designers to predict & evaluate alternative design.

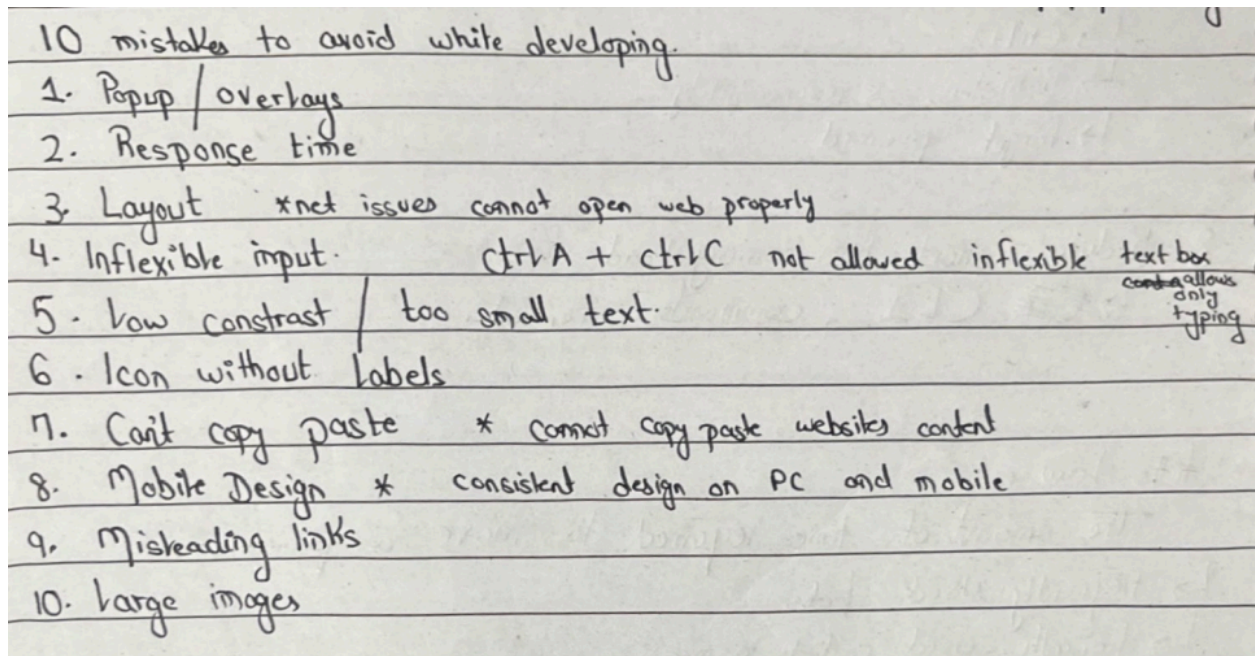
5. Frameworks set of Interrelated concepts and/or specific question for what to look for

HCI framework

- Designer's Model
- System Image
- The user's Model

top 10 mistakes in design

Mnemonic: CRIMP MILL



Overuse of Pop-ups/Overlays: If the app has frequent notifications, prompts, or promotional offers that interrupt the user experience, it can frustrate users, especially when these pop-ups are difficult to dismiss.

Slow Response Time: Users might experience lag when navigating the app, especially while performing critical transactions like money transfers or bill payments. Delays can lead to user dissatisfaction and even mistrust in completing sensitive transactions.

Layout Issues: If the app's layout doesn't adapt well to different network speeds or is broken on slower connections, users may face difficulties accessing features or reading important information.

Inflexible Input Fields: If the app restricts copy-paste functionality in fields (e.g., bank account numbers, transaction IDs) or doesn't allow users to use keyboard shortcuts like **Ctrl+C** for copying information, it can hinder convenience.

Low Contrast or Small Text: In cases where text fields or buttons are too small, or the contrast between text and background is too low, users with visual impairments or those using the app outdoors in bright light may struggle to read important information.

Icons without Labels: If the app uses icons that are not universally recognizable and doesn't provide text labels, new users might struggle to understand the function of certain buttons or features.

Inconsistent Mobile Design: If the design differs significantly between the mobile version and the web version of the app, users may find it hard to switch between platforms without a learning curve.

Limited Error Handling or Feedback: When errors occur (e.g., a failed transaction or input error), if the app doesn't provide clear and actionable feedback, users may be confused about what went wrong and how to fix it.

Misleading Links or Buttons: If buttons or links are placed in a confusing way or don't lead users where they expect, it could reduce trust in the app's navigation flow.

Large Image Files: If the app uses large image files for icons or background visuals, it may slow down load times on slower networks, leading to poor performance.

Prototyping

Definition:

Prototyping is the process of creating a preliminary version of a system or interface to explore ideas, test designs, and gather user feedback. It serves as a way to visualize and refine the design before full-scale development. Prototypes can range from simple sketches to interactive digital models.

Types of Prototyping:

1. Low-Fidelity Prototyping

Definition: Low-fidelity prototypes are simple, often non-interactive representations of a design. They are typically made using basic materials like paper or whiteboards and are easy and quick to create.

Example: A paper prototype of a mobile app that shows different screen layouts and user flow using hand-drawn sketches. The designer can walk users through the paper screens to gather initial feedback on the structure and layout.

Purpose: Low-fidelity prototypes are used for brainstorming and exploring different design ideas quickly. They are suitable for early-stage testing, where major design changes are still possible without significant investment.

2. High-Fidelity Prototyping

Definition: High-fidelity prototypes are more detailed and interactive representations

of the final product. They are often created using specialized design software (e.g., Figma, Adobe XD) and may include real data, refined visual elements, and interactions.

Example: A clickable prototype of a website created using Figma, where users can navigate through the interface, click on buttons, and experience transitions between pages as they would in the final product.

Purpose: High-fidelity prototypes are used for testing user interactions and fine-tuning design details. They are ideal for later stages of design, where feedback on the look and feel, as well as the user experience, is crucial.

Advantages of Prototyping:

- **Facilitates Early User Feedback:** By allowing users to interact with prototypes, designers can gather valuable feedback on usability, functionality, and design preferences before development begins.
- **Reduces Development Costs:** Identifying design flaws and usability issues in the prototyping stage prevents costly changes later in the development cycle.
- **Improves Communication:** Prototypes serve as a common visual reference for designers, developers, and stakeholders, helping align their understanding of the design.

Related Concepts:

- **Iterative Design:** Prototyping is closely linked with the iterative design process, where designs are repeatedly refined based on feedback from each round of testing.
- **Scenarios and Storyboards:** These are often used alongside low-fidelity prototypes to describe how users would interact with the system in different contexts.

Golden Ratio

- A ratio within the elements of a form, eg height to width, approx 0.618
- is often used as a guideline for creating visually appealing and balanced designs. This ratio can help designers create layouts, elements, and interfaces that are aesthetically pleasing and easier for users to engage with.
- Applications of the Golden Ratio in HCI:

1. Layout Design (eg, If the main content area is 1000 pixels wide, the sidebar might be approximately 618 pixels wide, while the footer could occupy the remaining space in a way that feels balanced.)
2. Typography (eg, if the body text is set to 16 pixels, the heading could be set to approximately 26 pixels (16×1.618))
3. Visual Elements (eg, A button designed to be 100 pixels wide could be 61 pixels tall, following the Golden Ratio, to create a more pleasing visual hierarchy.)

Differences Between Multithreading and Multitasking in HCI

Aspect	Multithreading	Multitasking
Scope	Focuses on managing threads within a single process.	Focuses on managing multiple independent processes.
HCI Example	A music player app that downloads songs (thread 1) while playing music (thread 2).	A user edits a document while attending a video call.
User Experience Goal	Keeps the app responsive during complex tasks.	Allows seamless switching and parallel execution of apps.